

关于工程方向的学习文档+实现说明

前言

Stage -1 和Stage 0 包含了对于在这个项目开发的过程中所需要学习的前提知识。

Stage 1、 Stage 2、 Stage3包括了对于实现过程中所学习到的知识以及开发过程的思路以及心得体会。

Stage -1 版本控制工具git

这个就是每一个版本都会有记录，可以随时回溯到上一个版本，就不用自己再多创建多个重复性文件。由于在vscode等IDE中基本都存在git的图形化操作，所以下面只列出一些比较常见的Git命令，其他的可以详见“附件1：gitnotes.pdf”和“附件2：git工作流程.pdf”

```
1 git branch name
2 git branch -v
3 git checkout name
4 git merge name
5
6 git add
7 git commit -m logmessage
8
9 git push
10 git pull
11 git clone
```

Stage 0 知识储备

1.make工具的核心功能

- **避免重复构建**：通过时间戳明确依赖文件的构建的时间，判断是否又新的更改，从而决定是否需要重新构建，节省了时间。
- **便携的多文件编译功能**：在项目开发的过程中，难免会遇到需要多文件在同一个项目编译的需求。如果文件较少，还可以通过基本的命令行语句实现构建，但是如果文件过多，不论是在刚开始的构建还是在后期的修改都需要耗费大量的时间，make工具就完美的避免了这个，实现了一劳永逸；同时make工具还能够自动识别各个文件之间的依赖关系，完成自动编译的功能。

2.makefile的基础语法规则

makefile文件其实是告诉make工具如何工作的命令文件，所以现在要使用make工具，就需要与它沟通的“语言”就是makefile,所以现在我们开始讨论makefile的语法规则。

2.1 makefile三要素：

- **目标**：make之后会生成什么。eg: main main.o
- **依赖（先决条件）**：告诉make如何实现目标的生成。eg: head1.c head1.h
- **命令**：可以是bash中shell命令。

2.2 make的工作流程：

- 确定需要最终的目标
- 递归检查依赖树，同时判断是否需要修改，重新构建（时间戳的应用）。
- 最后从底部自下而上一步步构建最终的目标。

2.3 简单的makefile语法：

2.3.1 目标：依赖

命令

(在这里需要注意的是命令的前面需要一个TAB键)

- eg:

```
1 all:main.o foo.o
2     gcc -o simple main.o foo.o
3 main.o:
4     gcc -o main.o -c main.c
5 foo.o:
6     gcc -o foo.o -c foo.c
7 clean:
8     rm simple main.o foo.o
```

2.3.2 clean目标：

用于删除生成的文件，我觉得就类似于一个定义。把rm删除命令用clean代替。

运行时只需要输入命令行命令：

```
1 make clean
```

2.3.3 伪对象.PHONY：

在实际的开发过程难免会遇到目标的名字和文件的名字一样的情况所以我们需要的做的是告诉make工具这是一个目标，就会用到.PHONY

```
1 .PHONY:clean
2 all:main.o foo.o
3     gcc -o simple main.o foo.o
4 main.o:
5     gcc -o main.o -c main.c
6 foo.o:
7     gcc -o foo.o -c foo.c
8 clean:
9     rm simple main.o foo.o
```

2.3.4 变量：

- 变量的定义：

变量名=所期望的值

这之后对于变量的使用都需要用到\$(变量名)

```
1  .PHONY:clean
2  CC=gcc
3  RM=rm
4  EXE=simple
5  OBJS=main.o foo.o
6
7  $(EXE):$(OBJS)
8      $(CC) -o EXE $(OBJS)
9  main.o:
10     gcc -o main.o -c main.c
11  foo.o:
12     gcc -o foo.o -c foo.c
13  clean:
14     rm simple main.o foo.o
```

- 自动变量：

目标和先决条件的名字会在规则的命令中多次出现，如果修改了依赖的名，那就需要在命令中全部跟着改。这时候就需要用到Makefile中的自动变量（总的来说**自动变量一般是用在命令中的**）：

- \$@:

用于表示规则中的**目标**，当一个规则中有多个目标,这个所表示的就是其中任何造成命令被运行的目标。

- \$^:

表示规则中的**所有**先决条件。

- \$<:

表示规则中第一个先决条件。

现在我们来利用自动变量重写一下上面的例子：

```
1  .PHONY:clean
2  CC=gcc
3  RM=rm
4  EXE=simple
5  OBJS=main.o foo.o
6
7  $(EXE):$(OBJS)
8      CC -o $@ $^
9  main.o:main.c
10     CC -o $@ -c $^
11  foo.o:foo.c
12     CC -o $@ -c $^
13  clean:
14     RM $(EXE) $(OBJS)
```

- 特殊变量：

需要注意的是下面的两个特殊变量都是内部已经定义了的，建议不要重新重复定义。

- **MAKE:**

主要是当在另外一个Makefile中调用另外一个Makefile:

```
1 build_lib:
2     cd lib && $(MAKE)
3
4 build_src:
5     cd lib && $(MAKE)
```

- **MAKECMDGOALS:**

是用户在命令行实际传入的目标名，后面可能结合函数进行条件判断等。

- **变量的类别:**

- **递归扩展变量:**

对于一个“链”上的变量，后面的变量修改，会影响到前面的变量，“连锁反应”

```
1 CC=gcc
```

- **简单扩展变量:**

只对变量进行一次的展开赋值，后续的修改目标都不会影响到“链”上的其他变量。

```
1 CC:=gcc
```

- **条件赋值变量:**

仅当变量为定义时被赋值

```
1 CC?=gcc
```

- **追加赋值变量:**

添加值列表。

```
1 CC+=gcc
```

2.3.5 override指令:

使得用户无法在命令行修改变量的值:

```
1 ```makefile
2 override CC=gcc
3 ```
```

2.3.6 模式

将相同的后缀名进行相同的规则的目标作为一个整体的目标：

```
1 .PHONY:clean
2 CC=gcc
3 EXE=simple
4 OBJS=main.o foo.o
5 RM=rm
6
7 $(EXE):$(OBJS)
8     $(CC) -o $@ $^
9 %.o:%.c
10    $(CC) -o $@ -c $^
11 clean:
12    $(RM) $(EXE) $(OBJS)
```

2.3.7 函数

下面介绍几个函数：

2.3.7.1 addprefix函数

给字符串中的每个子串前加上一个前缀

```
1 $(addprefix prefix,names)
```

2.3.7.2 filter函数

从一个字符串中得到满足模式的字符串(在text中只保留pattern)：

```
1 $(filter pattern,text)
2 srcs:=$(filter %.c %.s,$(srcs))
```

2.3.7.3 filter-out函数

和filter函数作用相反，是剔除满足pattern的字符串。

2.3.7.4 patsubst函数

用来字符串替换的

```
1 $(patsubst pattern,replacement,text)
```

2.3.7.5 strip函数

除去变量中多余的空格。

```
1 $(strip string)
```

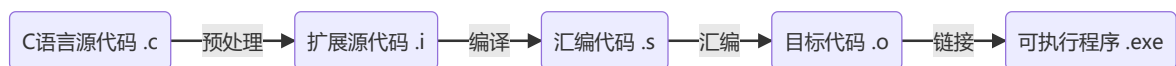
2.3.7.6 wildcard函数

得到所需的文件

```
1 | $(wildcard pattern)
```

```
1 | srcs=$(wildcard *.c)
```

3.C语言编译流程



- 预处理

- 删除注释
- 宏扩展

```
1 | #define
```

- 文件包含

```
1 | #include
```

- 编译

将.i的临时文件转化为汇编语言的汇编文件 (.s)

- 汇编

汇编文件汇编成计算机可以理解的二进制代码 (.o)

- 链接

将库文件.lib和.o一起成为最终的可执行文件.exe。但是也存在一种动态库不用直接复制到.exe文件中，而是运行时将.dll文件加载到内存，不用直接复制到.exe，减小.exe的体量。

4.C语言多文件开发的初步了解

4.1 分离代码

- 头文件 (.h)

- 声明函数原形，使得不同源文件之间可以调用这些函数。
- 定义宏和常量，便于在多个文件中共享使用。
- 声明全局变量，确保他们在不同的源文件中可见。

总之头文件的作用就是让函数，宏，变量等能够被多个文件共享。

```
1  #ifndef HEAD_H
2  #define HEAD_H
3
4  /*省略n行代码*/
5
6  #endif
```

注意上述的规范可以**避免重复包含所导致的重复定义**

- **源文件 (.c)**
 - 实现头文件中声明的函数
 - 定义主要的程序逻辑 (main.c)

4.2 模块化编程

每一个模块实现各自的功能，每个模块尽量代码量小，结构清晰，便于阅读理解

4.3 项目管理

- **版本控制**
 - git
- **项目结构**

```
1  project/
2  |
3  ├── include/
4  |   ├── math_util.h
5  |   └── file_util.h
6  |
7  ├── src/
8  |   ├── math_util.c
9  |   └── file_util.c
10 |
11 └── main.c
```

4.4 减少编译时间

- **增量编译**

只编译修改过的文件不需要编译整个文件，减少编译时间，提高开发效率
- **Makefile**

Stage 1：基础框架搭建（基础IO与文件操作）

mission1:命令行参数解析

- **关于argc和argv获取用户输入参数的学习**
 - argc: argument count 参数个数
 - argv: argument value 参数值

在命令行窗口通过cd命令进入可执行程序所在的目录下后输入依次输入：text.exe arg1 arg2便会运行程序的同时传入三个参数分别为可执行程序的路径和名称，以及后面输入的实际参数。

- 对于实现命令识别功能（实现-help帮助信息）

要求：能正确处理参数确实、格式错误等异常情况

检查点：

- 能够用识别-help的存在，输出里包含字段用法或Usage报价函-help参数
- 能够识别不正确的参数，例如-fenihfeihfeiu这样的混乱的字段

mission2:预处理与文件读取

- ctype.h的使用：

```
1 // 检查字符类型的函数
2 isspace(c) // 检查是否是空白字符（空格、制表符、换行等）
3 isalpha(c) // 检查是否是字母
4 isdigit(c) // 检查是否是数字
5 isalnum(c) // 检查是否是字母或数字
```

mission3:静态语法检查

Stage 2：解析器的开发（字符串与结构体）

- 关于解析器的定义

解析器（Parser），一般是指把某种格式的文本（字符串）转换成某种数据结构的过程。

- 解析器的作用

编译器无法直接操作字符串，代码是以字符串形式呈现的一种码，所以对于程序语言，解码的动作叫做parsing，用于解码的那段代码就是parser。解析器就是把字符串解码成数据结构。

- 在编译器中的功能

- 词法分析
把文本切割为单词的流
- 语法分析
输出语法树

mission2:关于任务2的收获

在实现minimake的过程理解了make的大致的简单的工作流程：

- 解析Makefile(Parse)

- 预处理
- 构建数据结构(rule)

- 静态依赖检查(Check)

在minimake中只实现了一些简单的检查

- 选择目标与执行命令(Run)

- 目标的选择：
类似于make run, make clean即选择要先构建哪个目标，不输入参数就是默认执行第一个目标，所以一般第一个目标都是最终的可执行程序。
- 执行命令

就是按照顺序打印到system(cmd)中执行目标下的每条命令。

Stage 3: 依赖关系排序

- 关于任务1和任务2的初步的分析

- 依赖的关系，其实本质就是一个拓扑关系，而拓扑图本身是一个有向无环图，由于不存在环，注定会导致这个图很稀疏，所以将会采取邻接表来存储这个图，因为如果采取邻接矩阵的话会有很多为0的点，浪费空间。
- 任务2是关于拓扑排序，所以需要用到入度，所以在第一个任务中也就是图的构建过程中，就顺带计算每个顶点的入度
- 任务2拓扑排序（Kahn算法）就可以用入度和一个栈（也可以是队列，主要是需要一个容器来存放入度为0的顶点）来解决即可，先找图中入度为0的顶点作为第一个，把他取出来之后，与他邻接的每一个顶点入度减一，然后把这次操作中产生的入度为0的顶点存储到栈中，然后接下来再从栈中取顶点，直到最后所有节点都被取出。
- 在任务2中寻找入度为0的顶点，难免会遇到图搜索的算法，也就涉及到深度优先搜索和广度优先搜索。搜索算法的关键在于visited数组也就是该顶点是否被访问过，虽然在拓扑图中不会出现环，但是由于采用的是邻接表来存储这个图，所以一定会有重复的顶点。

- 关于任务3的分析

时间戳（Unix Timestamp）是从 1970年1月1日 00:00:00 UTC 开始经过的秒数，用于表示一个时间点。

- 在makefile中的时间戳比较逻辑

```
1 | 如果 目标文件不存在 → 需要构建
2 | 否则 如果 任何依赖文件比目标文件新 → 需要构建
3 | 否则 → 跳过构建（目标已经是最新的）
```

- “构建系统的自举”

能够用正在开发的minimake工具成功编译他自己

自举的意义：

- 功能验证
- 实际测试
- 质量标志

- 对于Stage 3的总结：

为了实现任务三，需要把整个Stage 3整合在一起，而在实现Minimake自举的过程就是基本的Make的流程：

- 关键的数据结构：

Rule：包括了目标，依赖和命令。为一个整体的数据结构。

- 解析规则：

从MiniMakefile中读取相应的规则，解析到Rule数据结构中。在解析的过程中也实现一些基本的静态语法检查。

- 构建依赖关系的图：

这个图是一个有向无环图，当然也有可能会出现多个独立的子图，所以为了避免出现多余的构建，可以有一个辅助数组needed辅助判断是否需要构建。

- 拓扑排序：

构建的过程涉及到了依赖，也就是谁是谁的前提，决定先构建谁的问题就是拓扑排序。所以用到kahn算法实现拓扑排序。

- **时间戳检查:**

根据时间戳的检查逻辑（见上），判断是否需要构建。

- **执行命令:**

经过时间戳检查，如果需要构建或者重新构建就执行命令。

这样就是Make的主要的执行流程，同时也是MiniMake的实现逻辑。

参考文献

[1] Deepseek. DeepSeek-R1-0528. 深度求索(DeepSeek). 2025.09.

[2] GPT. GPT-5. OpenAI. 2025.09.

[3] Lion 莱恩呀. [轻松掌握Makefile基础知识：从入门到实战，一步步学习Makefile基础用法](#). 开发者社区. 2025-09-13.

[4] Lion-Zwart. [C语言的编译过程详解](#). 知乎. 2022-09-01.

[5] Edit2. [C语言如何多文件编程](#). PingCode. 2024-08-29.

[6] pl0020. [argc和argv，参数个数和参数值](#). CSDN. 2020-06-08.

[7] xwy7977. [解析器parser](#). 知乎. 2021-10-31.

[8] 严蔚敏. 数据结构(C语言版). 清华大学出版社. 2021.